

Projeto Motiva — Atualizacao Remota de Firmware (OTA)

S2-CP02 · Professor Marcelo Fernando Morgantini

No IoT de monitoramento de vegetacao em ESP32 simulado no Wokwi

21/09/2026

1. Integrantes

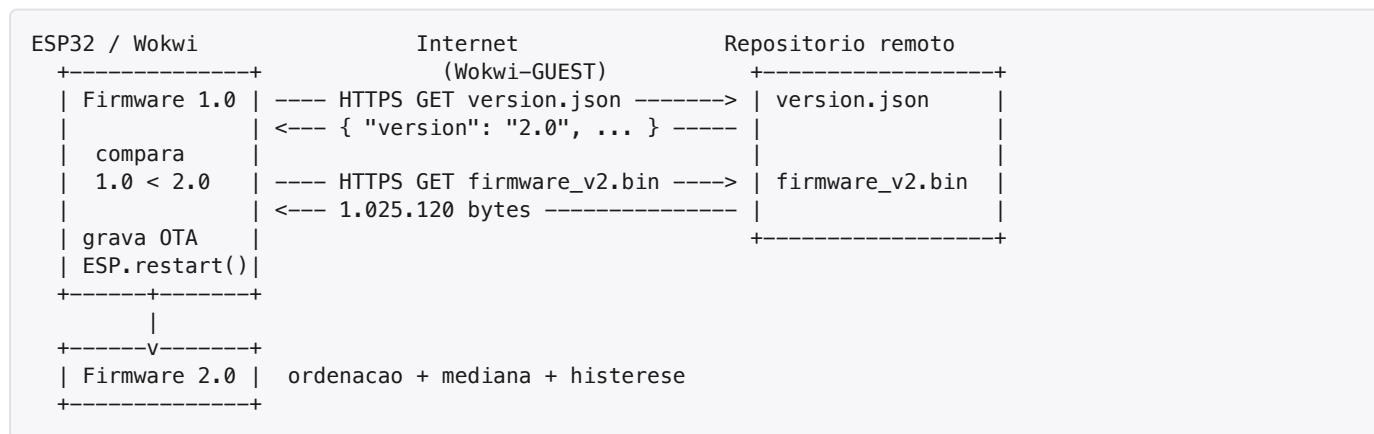
Nome	RM
Helena Barbosa Costa	562450
Henrique Mandrick	562715
Mateus Scandiuzzi Valente Tomomitsu	561565
Ryan Amorim de Castro Santana	564393
Thomas Joh Kobayashi	562758

2. Links da entrega

Item	Link
Projeto Wokwi (publico)	https://wokwi.com/projects/475816561802161153
Repositorio remoto (OTA)	https://github.com/henrikmm/motiva-ota-cp2
Manifesto de versao	https://raw.githubusercontent.com/henrikmm/motiva-ota-cp2/main/version.json
Binario da versao 2.0	https://raw.githubusercontent.com/henrikmm/motiva-ota-cp2/main/firmware_v2.bin

A estrutura permanece ativa e publica por, no minimo, 10 dias para validacao.

3. Arquitetura da solucao



Fluxo: consultar versao → comparar → baixar .bin → atualizar → reiniciar.

Particionamento e por que o OTA funciona

O ESP32 é compilado no esquema **Default 4MB with spiffs**, que reserva duas partições de aplicação de **1.310.720 bytes** (app0 e app1). O firmware em execução ocupa uma delas; a biblioteca HTTPUpdate grava o binário recebido na outra e marca, na partição otadata, qual deve ser carregada no próximo boot. Por isso o `ESP.restart()` já sobe na versão 2.0.

O `firmware_v2.bin` ocupa **1,025,120 bytes (78%)** da partição. O `build.sh` falha propositalmente se esse limite for ultrapassado.

Detalhe observado na demonstração: após o `ESP.restart()` do OTA, o contador de `millis()` do Firmware 2.0 não reiniciou em zero (ver seção 9): ele continuou a partir de onde estava antes do reset. Isso é comportamento real do ESP32 — um reset por software (`SW_CPU_RESET`) reinicia a CPU mas não zera o domínio de tempo da RTC, diferente de um power-on reset (`POWERON_RESET`), que zera tudo. O log confirma os dois tipos de reset nos momentos certos.

Hardware simulado

Componente	Ligação
ESP32 DevKit C v4	-
LED RGB (catodo comum) - canal R	GPIO 25 -> resistor 220 ohm
LED RGB (catodo comum) - canal G	GPIO 26 -> resistor 220 ohm
LED RGB (catodo comum) - canal B	GPIO 27 -> resistor 220 ohm
LED RGB - COM	GND

Indicação por LED

Situação	Cor
Firmware 1.0 em execução	Azul
Download/gravação OTA em andamento	Magenta
Firmware 2.0 - estado NORMAL	Verde
Firmware 2.0 - estado ALERTA	Vermelho

4. Temporização das sessões

O enunciado exige que a nova sessão comece **48 s após o início da anterior**, e não 48 s depois da quinta leitura:

```
00 s -> leitura 1      (5 leituras x 2 s = 8 s de aquisição)
02 s -> leitura 2
04 s -> leitura 3
06 s -> leitura 4
08 s -> leitura 5
48 s -> início da sessão seguinte    <- e não 56 s
```

A implementação **não usa `delay()` longo**. Cada sessão, ao começar, já agenda a próxima em `tInicioSessão + 48000`, e o `loop()` compara o relógio com `millis()`. Toda comparação usa `(int32_t)(millis() - alvo) >= 0`, forma que continua correta quando `millis()` estoura em 32 bits (a cada ~49 dias).

Cada cabeçalho de sessão imprime o instante e o intervalo medido desde a sessão anterior, o que comprova o requisito diretamente no Serial Monitor.

5. Conceitos aplicados

Media

Soma das 5 leituras dividida por 5. E sensível a valores extremos: uma única leitura muito alta desloca a média inteira.

Mediana

Valor central do vetor **ordenado**. Com 5 leituras, é o terceiro elemento (ordenado [2]). A ordenação é feita no próprio programa, por **inserção**, sobre uma cópia do vetor — a ordem original é preservada para exibição.

A mediana é usada na decisão de estado por ser **robusta a outliers**: uma leitura espúria de 20 cm no meio de um canteiro de 12 cm move a média, mas não move a mediana. Em um sensor de campo, isso evita alarmes falsos.

Histerese

Condição	Comportamento
mediana $\geq$ 16 cm	entra em ALERTA
14 cm < mediana < 16 cm	mantém o estado anterior
mediana $\leq$ 14 cm	entra/retorna para NORMAL

Sem histerese, uma vegetação oscilando entre 15 e 16 cm faria o LED piscar entre verde e vermelho a cada sessão. Com a faixa morta, o estado só muda quando a medida ultrapassa uma das bordas de forma decisiva.

Observação sobre o enunciado: o "exemplo de comportamento" da seção 9 cita medianas de 14,5 cm. Como as leituras são inteiras (10 a 20 cm), a mediana de cinco valores é **sempre inteira** — a faixa de manutenção contém unicamente a mediana **15 cm**. A implementação segue a tabela normativa, que é a regra.

Por que OTA é útil em campo

Os nós da Motiva ficam instalados em áreas de difícil acesso, por vezes a quilômetros de estrada. Sem OTA, corrigir um cálculo errado ou mudar um limiar exigiria deslocar uma equipe até cada equipamento, com custo de viagem, risco operacional e janela de indisponibilidade. Com OTA, a correção é publicada uma vez no repositório e todos os nós se atualizam sozinhos na próxima janela de comunicação.

6. Bibliotecas utilizadas

Biblioteca	Origem	Função no projeto
WiFi.h	core ESP32	conecta a rede virtual Wokwi-GUEST
WiFiClientSecure.h	core ESP32	socket TLS para falar com o GitHub via HTTPS
HTTPClient.h	core ESP32	requisição GET do version.json
HTTPUpdate.h	core ESP32	baixa o .bin e grava na partição OTA inativa

Nenhuma biblioteca de terceiros é usada. O `version.json` tem formato fixo e conhecido, então é lido por uma função própria (`extrairValorJson`) em vez de um parser JSON externo — isso elimina uma dependência, reduz o binário e mantém o projeto compilável no Wokwi sem configuração adicional.

A comparação de versões é **numérica**, não textual: "2.0" vira 200 e "1.0" vira 100. Comparar strings quebraria em um futuro "10.0", que é textualmente menor que "2.0".

7. Tratamento de situacoes previstas

Situacao	Mensagem no Serial Monitor
Sem conexao Wi-Fi	[OTA] ERRO: nao foi possivel conectar ao Wi-Fi.
Manifesto inacessivel	[OTA] ERRO: manifesto inacessivel (HTTP <codigo>).
Manifesto malformado	[OTA] ERRO: manifesto sem os campos "version" e "url".
Ja esta na versao mais recente	[OTA] Firmware ja esta na versao mais recente.
Download ou gravacao falhou	[OTA] ERRO <n> durante a atualizacao: <descricao>

Em todos os casos de falha o no **continua medindo normalmente** na versao atual. Uma atualizacao que nao deu certo nunca derruba o servico de medicao.

8. Testes obrigatorios

Teste	Condicao	Resultado esperado
1	Firmware 1.0	5 leituras, media e LED da versao 1.0
2	Sessao completa	Nova sessao iniciando em 48 s, e nao 56 s
3	Manifesto indica versao 2.0	ESP32 identifica atualizacao disponivel, apos 3 ciclos
4	OTA executada	ESP32 reinicia executando Firmware 2.0
5	Firmware 2.0	Media, ordenacao e mediana exibidas corretamente
6	Mediana ≥ 16 cm	Estado ALERTA e indicacao vermelha
7	Mediana entre 14 e 16 cm	Estado anterior mantido
8	Mediana ≤ 14 cm	Estado NORMAL e indicacao verde

9. Evidencias

Metodologia de validacao: a fila de compilacao gratuita do editor web do Wokwi apresentou instabilidade recorrente ("Build Servers Busy" / "Failed to fetch") durante os testes. Para nao depender dela, o grupo compilou os dois firmwares localmente com o `arduino-cli` (o mesmo binario que e publicado no repositorio) e executou a simulacao completa pela **Wokwi CLI** (`wokwi-cli`), que usa o mesmo motor de simulacao em nuvem do editor web, valida o `diagram.json` e reproduz fielmente Wi-Fi, HTTPS e a gravacao OTA. O log abaixo e a saida real e integral do Serial Monitor dessa execucao, sem edicao de conteudo (apenas uma quebra de linha reconstituída onde a captura uniu duas mensagens). O projeto publicado no link da secao 2 roda o mesmo codigo e pode ser reexecutado a qualquer momento clicando em Play.

03 led azul fw1

01_serial_fw1_fluxo_ota_completo.txt

```
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:4876
ho 0 tail 12 room 4
load:0x40078000,len:16532
load:0x40080400,len:3500
entry 0x400805b4
```

```
#####
  PROJETO MOTIVA - FIRMWARE 1.0
  No de monitoramento de vegetacao
  Verificacao OTA apos 3 sessoes
#####
=====
```

MONITORAMENTO DE VEGETACAO - FW 1.0

Sessao 1 | t = 1.75 s | intervalo desde a sessao anterior: 0.00 s
Leitura 1: 17 cm
Leitura 2: 14 cm
Leitura 3: 16 cm
Leitura 4: 17 cm
Leitura 5: 11 cm
Media da sessao: 15.0 cm
Proxima sessao em 48 segundos.

MONITORAMENTO DE VEGETACAO - FW 1.0

Sessao 2 | t = 49.75 s | intervalo desde a sessao anterior: 48.00 s
Leitura 1: 16 cm
Leitura 2: 18 cm
Leitura 3: 12 cm
Leitura 4: 16 cm
Leitura 5: 15 cm
Media da sessao: 15.4 cm
Proxima sessao em 48 segundos.

MONITORAMENTO DE VEGETACAO - FW 1.0

Sessao 3 | t = 97.75 s | intervalo desde a sessao anterior: 48.00 s
Leitura 1: 10 cm
Leitura 2: 16 cm
Leitura 3: 17 cm
Leitura 4: 18 cm
Leitura 5: 20 cm
Media da sessao: 16.2 cm
Proxima sessao em 48 segundos.

VERIFICACAO DE ATUALIZACAO (apos 3 sessoes) Versao instalada: 1.0

[OTA] Conectando a rede "Wokwi-GUEST"...

.....

[OTA] Wi-Fi conectado. IP: 10.13.37.2

[OTA] Consultando manifesto: <https://raw.githubusercontent.com/henrikmm/motiva-ota-cp2/main/version.json>

[OTA] Manifesto recebido:

```
{
  "version": "2.0",
  "url": "https://raw.githubusercontent.com/henrikmm/motiva-ota-cp2/main/firmware_v2.bin",
  "notes": "Ordenacao crescente das leituras, calculo da mediana e histerese NORMAL/ALERTA com indicacao por LED RGB.",
  "size": 1025120,
  "sha256": "f33e883e79e446a1a9c4ea5ce4f04276d3b8a45f8f524ef7df4b90182943c527"
}
```

[OTA] Versao instalada: 1.0 | versao disponivel: 2.0

[OTA] Atualizacao disponivel. Iniciando download...

[OTA] Baixando firmware: https://raw.githubusercontent.com/henrikmm/motiva-ota-cp2/main/firmware_v2.bin

[OTA] Download: 0% (0 / 1025120 bytes)

[OTA] Download: 10% (106496 / 1025120 bytes)

[OTA] Download: 20% (208896 / 1025120 bytes)

[OTA] Download: 30% (311296 / 1025120 bytes)

[OTA] Download: 40% (413696 / 1025120 bytes)

[OTA] Download: 50% (516096 / 1025120 bytes)

[OTA] Download: 60% (618496 / 1025120 bytes)

[OTA] Download: 70% (720896 / 1025120 bytes)

[OTA] Download: 80% (823296 / 1025120 bytes)

[OTA] Download: 90% (925696 / 1025120 bytes)

[OTA] Download: 100% (1025120 / 1025120 bytes)

[OTA] Gravacao concluida com sucesso.

[OTA] Reiniciando o ESP32 para executar o Firmware 2.0...

ets Jul 29 2019 12:21:46

rst:0xc (SW_CPU_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)

configsip: 0, SPIWP:0xee

clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00

mode:DIO, clock div:1

load:0x3fff0030,len:4876

ho 0 tail 12 room 4

load:0x40078000,len:16532

load:0x40080400,len:3500

entry 0x400805b4

#####

PROJETO MOTIVA - FIRMWARE 2.0

Atualizacao OTA concluida com sucesso

Novos recursos: ordenacao, mediana e histerese

```

Comandos de teste: T1 / T2 / T3
#####

-----
VERIFICACAO DE VERSAO APOS O BOOT
Versao instalada: 2.0

-----
[OTA] Conectando a rede "Wokwi-GUEST"...
.....
[OTA] Wi-Fi conectado. IP: 10.13.37.2
[OTA] Versao instalada: 2.0 | versao disponivel: 2.0
[OTA] Firmware ja esta na versao mais recente. Nenhuma acao necessaria.

=====
MONITORAMENTO DE VEGETACAO - FW 2.0
=====
Sessao 1 | t = 158.08 s | intervalo desde a sessao anterior: 0.00 s
Leitura 1: 10 cm
Leitura 2: 16 cm
Leitura 3: 13 cm
Leitura 4: 19 cm
Leitura 5: 15 cm
Valores na ordem original : 10 16 13 19 15 cm
Valores em ordem crescente: 10 13 15 16 19 cm
Media da sessao: 14,6 cm
Mediana da sessao: 15 cm
Histerese: 14 cm < mediana < 16 cm -> faixa de histerese, estado mantido
Estado anterior: NORMAL | Estado atual: NORMAL
LED: VERDE
Proxima sessao em 48 segundos.

```

02_serial_fw2_histerese_T1_T2_T3_T2.txt

```

ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:4876
ho 0 tail 12 room 4
load:0x40078000,len:16532
load:0x40080400,len:3500
entry 0x400805b4

#####
PROJETO MOTIVA - FIRMWARE 2.0
Atualizacao OTA concluida com sucesso
Novos recursos: ordenacao, mediana e histerese
Comandos de teste: T1 / T2 / T3
#####

-----
VERIFICACAO DE VERSAO APOS O BOOT
Versao instalada: 2.0

-----
[OTA] Conectando a rede "Wokwi-GUEST"...
.....
[OTA] Wi-Fi conectado. IP: 10.13.37.2
[OTA] Versao instalada: 2.0 | versao disponivel: 2.0
[OTA] Firmware ja esta na versao mais recente. Nenhuma acao necessaria.

=====
MONITORAMENTO DE VEGETACAO - FW 2.0
=====
Sessao 1 | t = 6.53 s | intervalo desde a sessao anterior: 0.00 s
Leitura 1: 15 cm
Leitura 2: 19 cm
Leitura 3: 10 cm
Leitura 4: 20 cm
Leitura 5: 18 cm
Valores na ordem original : 15 19 10 20 18 cm
Valores em ordem crescente: 10 15 18 19 20 cm
Media da sessao: 16.4 cm
Mediana da sessao: 18 cm
Histerese: mediana >= 16 cm -> entra em ALERTA
Estado anterior: NORMAL | Estado atual: ALERTA
LED: VERMELHO
Proxima sessao em 48 segundos.

-----
SESSAO DE TESTE INJETADA - T1: mediana >= 16 cm

```

Leitura 1: 18 cm
Leitura 2: 16 cm
Leitura 3: 17 cm
Leitura 4: 20 cm
Leitura 5: 16 cm
Valores na ordem original : 18 16 17 20 16 cm
Valores em ordem crescente: 16 16 17 18 20 cm
Media da sessao: 17.4 cm
Mediana da sessao: 17 cm
Histerese: mediana >= 16 cm -> entra em ALERTA
Estado anterior: ALERTA | Estado atual: ALERTA
LED: VERMELHO

SESSAO DE TESTE INJETADA - T2: 14 cm < mediana < 16 cm

Leitura 1: 13 cm
Leitura 2: 15 cm
Leitura 3: 20 cm
Leitura 4: 15 cm
Leitura 5: 12 cm
Valores na ordem original : 13 15 20 15 12 cm
Valores em ordem crescente: 12 13 15 15 20 cm
Media da sessao: 15.0 cm
Mediana da sessao: 15 cm
Histerese: 14 cm < mediana < 16 cm -> faixa de histerese, estado mantido
Estado anterior: ALERTA | Estado atual: ALERTA
LED: VERMELHO

SESSAO DE TESTE INJETADA - T3: mediana <= 14 cm

Leitura 1: 10 cm
Leitura 2: 14 cm
Leitura 3: 13 cm
Leitura 4: 12 cm
Leitura 5: 14 cm
Valores na ordem original : 10 14 13 12 14 cm
Valores em ordem crescente: 10 12 13 14 14 cm
Media da sessao: 12.6 cm
Mediana da sessao: 13 cm
Histerese: mediana <= 14 cm -> entra/retorna para NORMAL
Estado anterior: ALERTA | Estado atual: NORMAL
LED: VERDE

SESSAO DE TESTE INJETADA - T2: 14 cm < mediana < 16 cm

Leitura 1: 13 cm
Leitura 2: 15 cm
Leitura 3: 20 cm
Leitura 4: 15 cm
Leitura 5: 12 cm
Valores na ordem original : 13 15 20 15 12 cm
Valores em ordem crescente: 12 13 15 15 20 cm
Media da sessao: 15.0 cm
Mediana da sessao: 15 cm
Histerese: 14 cm < mediana < 16 cm -> faixa de histerese, estado mantido
Estado anterior: NORMAL | Estado atual: NORMAL
LED: VERDE

10. Arquivo version.json

```
{
  "version": "2.0",
  "url": "https://raw.githubusercontent.com/henrikmm/motiva-ota-cp2/main/firmware_v2.bin",
  "notes": "Ordenacao crescente das leituras, calculo da mediana e histerese NORMAL/ALERTA com indicacao por LED RGB.",
  "size": 1025120,
  "sha256": "f33e883e79e446a1a9c4ea5ce4f04276d3b8a45f8f524ef7df4b90182943c527"
}
```

11. Arquivo firmware_v2.bin

O binario nao pode ser embutido em PDF. Ele esta publicado no repositorio e e o arquivo efetivamente baixado pelo ESP32 durante a demonstracao. Sua identidade e comprovada pelo hash:

Atributo	Valor
URL	https://raw.githubusercontent.com/henrikmm/motiva-ota-cp2/main/firmware_v2.bin
Tamanho	1.025.120 bytes
SHA-256	f33e883e79e446a1a9c4ea5ce4f04276d3b8a45f8f524ef7df4b90182943c527
Particao de destino	app0/app1 (1.310.720 bytes) - esquema Default 4MB with spiiffs

Para conferir apos o download: `shasum -a 256 firmware_v2.bin`

12. Código-fonte completo — Firmware 1.0

```
1  /* =====
2  S2-CP02 - Projeto Motiva | Atualizacao Remota de Firmware (OTA)
3  FIRMWARE 1.0 - No de sensoriamento de vegetacao (ESP32 / Wokwi)
4
5  Integrantes:
6      Helena Barbosa Costa          RM 562450
7      Henrique Mandrick            RM 562715
8      Mateus Scandiuizzi Valente Tomomitsu RM 561565
9      Ryan Amorim de Castro Santana  RM 564393
10     Thomas Joh Kobayashi          RM 562758
11
12  Funcao desta versao:
13     - 5 leituras pseudoaleatorias (10 a 20 cm) por sessao, uma a cada 2 s
14     - media aritmetica da sessao
15     - nova sessao a cada 48 s contados do INICIO da sessao anterior
16     - LED AZUL indicando "firmware 1.0 em execucao"
17     - apos 3 sessoes, consulta o manifesto remoto e, havendo versao mais
18       nova, baixa o .bin e executa a atualizacao OTA
19  ===== */
20
21 #include <WiFi.h>
22 #include <WiFiClientSecure.h>
23 #include <HTTPClient.h>
24 #include <HTTPUpdate.h>
25
26 /* ----- IDENTIDADE */
27 static const char *FW_VERSION = "1.0";
28
29 /* ----- REDE */
30 static const char *WIFI_SSID = "Wokwi-GUEST";
31 static const char *WIFI_PASS = "";
32 static const uint8_t WIFI_CANAL = 6; /* acelera o connect no Wokwi */
33 static const uint32_t WIFI_TIMEOUT_MS = 20000UL;
34
35 static const char *MANIFEST_URL =
36     "https://raw.githubusercontent.com/henrikmm/motiva-ota-cp2/main/version.json";
37
38 /* ----- HARDWARE */
39 /* LED RGB de catodo comum: nivel ALTO acende o canal */
40 static const uint8_t PIN_LED_R = 25;
41 static const uint8_t PIN_LED_G = 26;
42 static const uint8_t PIN_LED_B = 27;
43
44 /* ----- TEMPORIZACAO */
45 static const uint8_t LEITURAS_POR_SESSAO = 5;
46 static const uint32_t INTERVALO_LEITURA_MS = 2000UL; /* 2 s (2 min reais) */
47 static const uint32_t INTERVALO_SESSAO_MS = 48000UL; /* 48 s (48 h reais) */
48 static const uint8_t CICLOS_ANTES_OTA = 3;
49
50 /* ----- MEDICAO */
51 static const int ALTURA_MIN_CM = 10;
52 static const int ALTURA_MAX_CM = 20;
53
54 /* ----- ESTADO */
55 static int leituras[LEITURAS_POR_SESSAO];
56 static uint8_t indiceLeitura = 0;
57 static uint32_t tInicioSessao = 0;
58 static uint32_t tInicioSessaoAnterior = 0;
59 static uint32_t tProximaLeitura = 0;
60 static uint32_t tProximaSessao = 0;
61 static uint16_t sessoesConcluidas = 0;
62 static bool sessaoEmAndamento = false;
63 static bool otaJaTentada = false;
64
65 /* =====
66 COMPARACAO DE TEMPO SEGURA CONTRA OVERFLOW DE millis()
67 ===== */
68 static bool tempoAlcancado(uint32_t alvo) {
69     return (int32_t)(millis() - alvo) >= 0;
70 }
71
72 /* =====
73 LED
74 ===== */
75 static void configurarLed() {
76     pinMode(PIN_LED_R, OUTPUT);
77     pinMode(PIN_LED_G, OUTPUT);
78     pinMode(PIN_LED_B, OUTPUT);
```

```

79 }
80
81 static void definirCor(bool vermelho, bool verde, bool azul) {
82     digitalWrite(PIN_LED_R, vermelho ? HIGH : LOW);
83     digitalWrite(PIN_LED_G, verde ? HIGH : LOW);
84     digitalWrite(PIN_LED_B, azul ? HIGH : LOW);
85 }
86
87 /* Azul = firmware 1.0 em execucao (secao 10 do enunciado) */
88 static void indicarFirmwareAtivo() { definirCor(false, false, true); }
89
90 /* Magenta = processo de atualizacao OTA em andamento */
91 static void indicarAtualizacao() { definirCor(true, false, true); }
92
93 /* =====
94    AQUISICAO E ESTATISTICA
95    ===== */
96 static int gerarLeitura() {
97     return random(ALTURA_MIN_CM, ALTURA_MAX_CM + 1); /* 10..20 cm inclusive */
98 }
99
100 static float calcularMedia(const int valores[], uint8_t n) {
101     long soma = 0;
102     for (uint8_t i = 0; i < n; i++) soma += valores[i];
103     return (float)soma / (float)n;
104 }
105
106 /* =====
107    CICLO DE MEDICAO (nao bloqueante, baseado em millis)
108    ===== */
109 static void imprimirCabecalhoSessao() {
110     float agoraS = millis() / 1000.0f;
111     float delta = (tInicioSessaoAnterior == 0)
112                 ? 0.0f
113                 : (tInicioSessao - tInicioSessaoAnterior) / 1000.0f;
114
115     Serial.println();
116     Serial.println("=====");
117     Serial.printf("MONITORAMENTO DE VEGETACAO - FW %s\n", FW_VERSION);
118     Serial.println("=====");
119     Serial.printf("Sessao %u | t = %.2f s | intervalo desde a sessao anterior: %.2f s\n",
120                 (unsigned)(sessoesConcluidas + 1), agoraS, delta);
121 }
122
123 static void iniciarSessao() {
124     tInicioSessaoAnterior = tInicioSessao;
125     tInicioSessao = millis();
126     /* a proxima sessao e ancorada no INICIO desta, nao no fim da 5a leitura */
127     tProximaSessao = tInicioSessao + INTERVALO_SESSAO_MS;
128     tProximaLeitura = tInicioSessao;
129     indiceLeitura = 0;
130     sessaoEmAndamento = true;
131
132     imprimirCabecalhoSessao();
133 }
134
135 static void finalizarSessao() {
136     float media = calcularMedia(leituras, LEITURAS_POR_SESSAO);
137     Serial.printf("Media da sessao: %.1f cm\n", media);
138     Serial.printf("Proxima sessao em %lu segundos.\n", INTERVALO_SESSAO_MS / 1000UL);
139
140     sessaoEmAndamento = false;
141     sessoesConcluidas++;
142 }
143
144 static void processarSessao() {
145     if (!tempoAlcancado(tProximaLeitura)) return;
146
147     leituras[indiceLeitura] = gerarLeitura();
148     Serial.printf("Leitura %u: %d cm\n", (unsigned)(indiceLeitura + 1), leituras[indiceLeitura]);
149
150     indiceLeitura++;
151     tProximaLeitura += INTERVALO_LEITURA_MS;
152
153     if (indiceLeitura >= LEITURAS_POR_SESSAO) finalizarSessao();
154 }
155
156 /* =====
157    CONECTIVIDADE
158    ===== */
159 static bool conectarWiFi() {
160     if (WiFi.status() == WL_CONNECTED) return true;
161
162     Serial.printf("[OTA] Conectando a rede \"%s\"...\n", WIFI_SSID);

```

```

163 WiFi.mode(WIFI_STA);
164 WiFi.begin(WIFI_SSID, WIFI_PASS, WIFI_CANAL);
165
166 uint32_t limite = millis() + WIFI_TIMEOUT_MS;
167 while (WiFi.status() != WL_CONNECTED && !tempoAlcancado(limite)) {
168     delay(250);
169     Serial.print(".");
170 }
171 Serial.println();
172
173 if (WiFi.status() != WL_CONNECTED) {
174     /* Situacao prevista na secao 13: nao ha conexao Wi-Fi */
175     Serial.println("[OTA] ERRO: nao foi possivel conectar ao Wi-Fi.");
176     Serial.println("[OTA] Atualizacao cancelada. O no continua medindo com o FW 1.0.");
177     return false;
178 }
179
180 Serial.printf("[OTA] Wi-Fi conectado. IP: %s\n", WiFi.localIP().toString().c_str());
181 return true;
182 }
183
184 /* =====
185     MANIFESTO DE VERSAO
186     Leitura manual do JSON: o manifesto tem formato fixo e conhecido, o que
187     dispensa uma biblioteca de parsing externa e mantem o binario menor.
188     ===== */
189 static String extrairValorJson(const String &json, const String &chave) {
190     String alvo = "\"" + chave + "\"";
191     int p = json.indexOf(alvo);
192     if (p < 0) return "";
193     p = json.indexOf(':', p + alvo.length());
194     if (p < 0) return "";
195     int ini = json.indexOf('"', p);
196     if (ini < 0) return "";
197     int fim = json.indexOf('"', ini + 1);
198     if (fim < 0) return "";
199     return json.substring(ini + 1, fim);
200 }
201
202 static bool consultarManifesto(String &versaoRemota, String &urlFirmware) {
203     WiFiClientSecure cliente;
204     cliente.setInsecure();          /* Wokwi: sem validacao de cadeia de certificados */
205     cliente.setTimeout(20000);
206
207     HTTPClient http;
208     Serial.printf("[OTA] Consultando manifesto: %s\n", MANIFEST_URL);
209
210     if (!http.begin(cliente, MANIFEST_URL)) {
211         Serial.println("[OTA] ERRO: URL do manifesto invalida.");
212         return false;
213     }
214     http.setFollowRedirects(HTTPC_STRICT_FOLLOW_REDIRECTS);
215     http.setTimeout(20000);
216
217     int codigo = http.GET();
218     if (codigo != HTTP_CODE_OK) {
219         /* Situacao prevista na secao 13: o manifesto nao pode ser acessado */
220         Serial.printf("[OTA] ERRO: manifesto inacessivel (HTTP %d - %s).\n",
221             codigo, http.errorToString(codigo).c_str());
222         http.end();
223         return false;
224     }
225
226     String corpo = http.getString();
227     http.end();
228
229     Serial.println("[OTA] Manifesto recebido.");
230     Serial.println(corpo);
231
232     versaoRemota = extrairValorJson(corpo, "version");
233     urlFirmware = extrairValorJson(corpo, "url");
234
235     if (versaoRemota.length() == 0 || urlFirmware.length() == 0) {
236         Serial.println("[OTA] ERRO: manifesto sem os campos \"version\" e \"url\".");
237         return false;
238     }
239     return true;
240 }
241
242 /* Converte "2.0" em 200 para permitir comparacao numerica (e nao textual) */
243 static int versaoParaNumero(const String &versao) {
244     int ponto = versao.indexOf('.');
245     int maior = (ponto < 0) ? versao.toInt() : versao.substring(0, ponto).toInt();
246     int menor = (ponto < 0) ? 0 : versao.substring(ponto + 1).toInt();

```

```

247     return maior * 100 + menor;
248 }
249
250 static bool versaoEhMaisNova(const String &remota, const String &instalada) {
251     return versaoParaNumero(remota) > versaoParaNumero(instalada);
252 }
253
254 /* =====
255     ATUALIZACAO OTA
256     ===== */
257 static void aoProgreDir(int recebido, int total) {
258     static int ultimoPercentual = -1;
259     if (total <= 0) return;
260     int p = (recebido * 100) / total;
261     if (p != ultimoPercentual && p % 10 == 0) {
262         Serial.printf("[OTA] Download: %d%% (%d / %d bytes)\n", p, recebido, total);
263         ultimoPercentual = p;
264     }
265 }
266
267 static void executarAtualizacaoOTA(const String &urlFirmware) {
268     Serial.printf("[OTA] Baixando firmware: %s\n", urlFirmware.c_str());
269     indicarAtualizacao();
270
271     WiFiClientSecure cliente;
272     cliente.setInsecure();
273     cliente.setTimeout(30000);
274
275     httpUpdate.rebootOnUpdate(false); /* o reboot e feito por nos, apos o log */
276     httpUpdate.setFollowRedirects(HTTPC_STRICT_FOLLOW_REDIRECTS);
277     httpUpdate.onProgress(aoProgreDir);
278
279     t_httpUpdate_return resultado = httpUpdate.update(cliente, urlFirmware);
280
281     switch (resultado) {
282     case HTTP_UPDATE_OK:
283         Serial.println("[OTA] Gravacao concluida com sucesso.");
284         Serial.println("[OTA] Reiniciando o ESP32 para executar o Firmware 2.0...");
285         Serial.flush();
286         delay(500);
287         ESP.restart();
288         break;
289
290     case HTTP_UPDATE_NO_UPDATES:
291         /* Situacao prevista na secao 13: a versao instalada ja e a mais recente */
292         Serial.println("[OTA] Servidor informou que nao ha atualizacao a aplicar.");
293         indicarFirmwareAtivo();
294         break;
295
296     case HTTP_UPDATE_FAILED:
297     default:
298         /* Situacoes previstas na secao 13: download falhou / gravacao com erro */
299         Serial.printf("[OTA] ERRO %d durante a atualizacao: %s\n",
300             httpUpdate.getLastErrorCode(),
301             httpUpdate.getLastErrorString().c_str());
302         Serial.println("[OTA] O no permanece no FW 1.0 e continua medindo normalmente.");
303         indicarFirmwareAtivo();
304         break;
305     }
306 }
307
308 /* Orquestra todo o fluxo obrigatorio da secao 12 */
309 static void verificarAtualizacao() {
310     otaJaTentada = true;
311
312     Serial.println();
313     Serial.println("-----");
314     Serial.printf ("VERIFICACAO DE ATUALIZACAO (apos %u sessoes)\n", (unsigned)sessoesConcluidas);
315     Serial.printf ("Versao instalada: %s\n", FW_VERSION);
316     Serial.println("-----");
317
318     if (!conectarWiFi()) { indicarFirmwareAtivo(); return; }
319
320     String versaoRemota, urlFirmware;
321     if (!consultarManifesto(versaoRemota, urlFirmware)) { indicarFirmwareAtivo(); return; }
322
323     Serial.printf("[OTA] Versao instalada: %s | versao disponivel: %s\n",
324         FW_VERSION, versaoRemota.c_str());
325
326     if (!versaoEhMaisNova(versaoRemota, FW_VERSION)) {
327         /* Situacao prevista na secao 13: a versao instalada ja e a mais recente */
328         Serial.println("[OTA] Firmware ja esta na versao mais recente. Nenhuma acao necessaria.");
329         indicarFirmwareAtivo();
330         return;
331     }

```

```

331 }
332
333 Serial.println("[OTA] Atualizacao disponivel. Iniciando download...");
334 executarAtualizacaoOTA(urlFirmware);
335 }
336
337 /* =====
338     SETUP / LOOP
339     ===== */
340 void setup() {
341     Serial.begin(115200);
342     delay(300);
343
344     configurarLed();
345     indicarFirmwareAtivo();
346
347     randomSeed(esp_random());
348
349     Serial.println();
350     Serial.println("#####");
351     Serial.printf (" PROJETO MOTIVA - FIRMWARE %s\n", FW_VERSION);
352     Serial.println(" No de monitoramento de vegetacao");
353     Serial.printf (" Verificacao OTA apos %u sessoes\n", (unsigned)CICLOS_ANTES_OTA);
354     Serial.println("#####");
355
356     tProximaSessao = millis(); /* primeira sessao comeca imediatamente */
357 }
358
359 void loop() {
360     if (!sessaoEmAndamento && tempoAlcancado(tProximaSessao)) {
361         iniciarSessao();
362     }
363
364     if (sessaoEmAndamento) {
365         processarSessao();
366     }
367
368     if (!sessaoEmAndamento && !otaJaTentada && sessoesConcluidas >= CICLOS_ANTES_OTA) {
369         verificarAtualizacao();
370         /* a verificacao e bloqueante; se ela ultrapassou a janela de 48 s,
371            reancora o agendamento para nao disparar sessoes em rajada */
372         if (tempoAlcancado(tProximaSessao)) tProximaSessao = millis();
373     }
374 }

```

13. Código-fonte completo — Firmware 2.0

```
1  /* =====
2  S2-CP02 – Projeto Motiva | Atualizacao Remota de Firmware (OTA)
3  FIRMWARE 2.0 – No de sensoriamento de vegetacao (ESP32 / Wokwi)
4
5  Integrantes:
6      Helena Barbosa Costa          RM 562450
7      Henrique Mandrick            RM 562715
8      Mateus Scandiuizzi Valente Tomomitsu RM 561565
9      Ryan Amorim de Castro Santana RM 564393
10     Thomas Joh Kobayashi          RM 562758
11
12  Evolucao em relacao ao Firmware 1.0 (secao 7 do enunciado):
13      - mantem as 5 leituras / 2 s e o calculo da media
14      - ordena uma COPIA das leituras em ordem crescente (algoritmo proprio)
15      - exibe os valores na ordem original e na ordem crescente
16      - calcula e exibe a MEDIANA da sessao
17      - aplica HISTERESE sobre a mediana para alternar NORMAL / ALERTA
18      - LED VERDE = NORMAL, LED VERMELHO = ALERTA
19      - verifica o manifesto no boot e reporta que ja esta na versao mais recente
20
21  Comandos de teste pelo Serial Monitor (digite e tecle Enter):
22      T1 -> injeta sessao com mediana >= 16 cm (espera-se ALERTA / vermelho)
23      T2 -> injeta sessao com mediana = 15 cm (espera-se MANTER o estado)
24      T3 -> injeta sessao com mediana <= 14 cm (espera-se NORMAL / verde)
25  ===== */
26
27 #include <WiFi.h>
28 #include <WiFiClientSecure.h>
29 #include <HTTPClient.h>
30
31 /* ----- IDENTIDADE */
32 static const char *FW_VERSION = "2.0";
33
34 /* ----- REDE */
35 static const char *WIFI_SSID = "Wokwi-GUEST";
36 static const char *WIFI_PASS = "";
37 static const uint8_t WIFI_CANAL = 6;
38 static const uint32_t WIFI_TIMEOUT_MS = 20000UL;
39
40 static const char *MANIFEST_URL =
41     "https://raw.githubusercontent.com/henrikmm/motiva-ota-cp2/main/version.json";
42
43 /* ----- HARDWARE */
44 static const uint8_t PIN_LED_R = 25;
45 static const uint8_t PIN_LED_G = 26;
46 static const uint8_t PIN_LED_B = 27;
47
48 /* ----- TEMPORIZACAO */
49 static const uint8_t LEITURAS_POR_SESSAO = 5;
50 static const uint32_t INTERVALO_LEITURA_MS = 2000UL;
51 static const uint32_t INTERVALO_SESSAO_MS = 48000UL;
52
53 /* ----- MEDICAO */
54 static const int ALTURA_MIN_CM = 10;
55 static const int ALTURA_MAX_CM = 20;
56
57 /* ----- HISTERESE
58     Secao 9 do enunciado:
59     mediana >= 16 cm ..... entra em ALERTA
60     14 cm < mediana < 16 cm ..... mantem o estado anterior
61     mediana <= 14 cm ..... entra/retorna para NORMAL
62     Como as leituras sao numeros inteiros, a mediana de 5 valores tambem e
63     sempre inteira: a faixa de manutencao contem unicamente a mediana 15 cm. */
64 static const int LIMITE_ALERTA = 16;
65 static const int LIMITE_NORMAL = 14;
66
67 enum EstadoSistema { ESTADO_NORMAL, ESTADO_ALERTA };
68
69 /* ----- ESTADO */
70 static int leituras[LEITURAS_POR_SESSAO];
71 static uint8_t indiceLeitura = 0;
72 static uint32_t tInicioSessao = 0;
73 static uint32_t tInicioSessaoAnterior = 0;
74 static uint32_t tProximaLeitura = 0;
75 static uint32_t tProximaSessao = 0;
76 static uint16_t sessoesConcluidas = 0;
77 static bool sessaoEmAndamento = false;
78 static EstadoSistema estadoAtual = ESTADO_NORMAL;
```

```

79
80 /* =====
81     COMPARACAO DE TEMPO SEGURA CONTRA OVERFLOW DE millis()
82     ===== */
83 static bool tempoAlcancado(uint32_t alvo) {
84     return (int32_t)(millis() - alvo) >= 0;
85 }
86
87 /* =====
88     LED
89     ===== */
90 static void configurarLed() {
91     pinMode(PIN_LED_R, OUTPUT);
92     pinMode(PIN_LED_G, OUTPUT);
93     pinMode(PIN_LED_B, OUTPUT);
94 }
95
96 static void definirCor(bool vermelho, bool verde, bool azul) {
97     digitalWrite(PIN_LED_R, vermelho ? HIGH : LOW);
98     digitalWrite(PIN_LED_G, verde ? HIGH : LOW);
99     digitalWrite(PIN_LED_B, azul ? HIGH : LOW);
100 }
101
102 /* Secao 10: no FW 2.0 o LED passa a indicar o ESTADO do sistema */
103 static void atualizarLedPorEstado() {
104     if (estadoAtual == ESTADO_ALERTA) definirCor(true, false, false); /* vermelho */
105     else definirCor(false, true, false); /* verde */
106 }
107
108 static const char *nomeEstado(EstadoSistema e) {
109     return (e == ESTADO_ALERTA) ? "ALERTA" : "NORMAL";
110 }
111
112 static const char *nomeCorDoEstado(EstadoSistema e) {
113     return (e == ESTADO_ALERTA) ? "VERMELHO" : "VERDE";
114 }
115
116 /* =====
117     AQUISICAO E ESTATISTICA
118     ===== */
119 static int gerarLeitura() {
120     return random(ALTURA_MIN_CM, ALTURA_MAX_CM + 1);
121 }
122
123 static float calcularMedia(const int valores[], uint8_t n) {
124     long soma = 0;
125     for (uint8_t i = 0; i < n; i++) soma += valores[i];
126     return (float)soma / (float)n;
127 }
128
129 /* Ordenacao por insercao, implementada no proprio programa (secao 8).
130     Trabalha sobre uma COPIA para preservar a ordem original das leituras. */
131 static void ordenarCrescente(int destino[], const int origem[], uint8_t n) {
132     for (uint8_t i = 0; i < n; i++) destino[i] = origem[i];
133
134     for (uint8_t i = 1; i < n; i++) {
135         int chave = destino[i];
136         int j = (int)i - 1;
137         while (j >= 0 && destino[j] > chave) {
138             destino[j + 1] = destino[j];
139             j--;
140         }
141         destino[j + 1] = chave;
142     }
143 }
144
145 /* Para 5 valores, a mediana e o elemento central do vetor ordenado */
146 static int calcularMediana(const int ordenado[], uint8_t n) {
147     return ordenado[n / 2];
148 }
149
150 static void imprimirVetor(const char *rotulo, const int valores[], uint8_t n) {
151     Serial.print(rotulo);
152     for (uint8_t i = 0; i < n; i++) {
153         Serial.printf("%d", valores[i]);
154         if (i < n - 1) Serial.print(" ");
155     }
156     Serial.println(" cm");
157 }
158
159 /* =====
160     HISTERESE
161     ===== */
162 static EstadoSistema aplicarHisterese(int mediana, const char **motivo) {

```

```

163 if (mediana >= LIMITE_ALERTA) {
164     *motivo = "mediana >= 16 cm -> entra em ALERTA";
165     estadoAtual = ESTADO_ALERTA;
166 } else if (mediana <= LIMITE_NORMAL) {
167     *motivo = "mediana <= 14 cm -> entra/retorna para NORMAL";
168     estadoAtual = ESTADO_NORMAL;
169 } else {
170     *motivo = "14 cm < mediana < 16 cm -> faixa de histerese, estado mantido";
171     /* estadoAtual permanece inalterado */
172 }
173 return estadoAtual;
174 }
175
176 /* =====
177     PROCESSAMENTO DE UMA SESSAO COMPLETA
178     ===== */
179 static void processarResultadosDaSessao(const int valores[], uint8_t n) {
180     int ordenado[LEITURAS_POR_SESSAO];
181     ordenarCrescente(ordenado, valores, n);
182
183     float media = calcularMedia(valores, n);
184     int mediana = calcularMediana(ordenado, n);
185
186     EstadoSistema estadoAnterior = estadoAtual;
187     const char *motivo = "";
188     EstadoSistema estadoNovo = aplicarHisterese(mediana, &motivo);
189     atualizarLedPorEstado();
190
191     imprimirVetor("Valores na ordem original : ", valores, n);
192     imprimirVetor("Valores em ordem crescente: ", ordenado, n);
193     Serial.printf("Media da sessao: %.1f cm\n", media);
194     Serial.printf("Mediana da sessao: %d cm\n", mediana);
195     Serial.printf("Histerese: %s\n", motivo);
196     Serial.printf("Estado anterior: %s | Estado atual: %s\n",
197         nomeEstado(estadoAnterior), nomeEstado(estadoNovo));
198     Serial.printf("LED: %s\n", nomeCorDoEstado(estadoNovo));
199 }
200
201 /* =====
202     CICLO DE MEDICAO (nao bloqueante, baseado em millis)
203     ===== */
204 static void imprimirCabecalhoSessao() {
205     float agoraS = millis() / 1000.0f;
206     float delta = (tInicioSessaoAnterior == 0)
207         ? 0.0f
208         : (tInicioSessao - tInicioSessaoAnterior) / 1000.0f;
209
210     Serial.println();
211     Serial.println("=====");
212     Serial.printf("MONITORAMENTO DE VEGETACAO - FW %s\n", FW_VERSION);
213     Serial.println("=====");
214     Serial.printf("Sessao %u | t = %.2f s | intervalo desde a sessao anterior: %.2f s\n",
215         (unsigned)(sessoesConcluidas + 1), agoraS, delta);
216 }
217
218 static void iniciarSessao() {
219     tInicioSessaoAnterior = tInicioSessao;
220     tInicioSessao = millis();
221     tProximaSessao = tInicioSessao + INTERVALO_SESSAO_MS; /* ancorado no INICIO */
222     tProximaLeitura = tInicioSessao;
223     indiceLeitura = 0;
224     sessaoEmAndamento = true;
225
226     imprimirCabecalhoSessao();
227 }
228
229 static void finalizarSessao() {
230     processarResultadosDaSessao(leituras, LEITURAS_POR_SESSAO);
231     Serial.printf("Proxima sessao em %lu segundos.\n", INTERVALO_SESSAO_MS / 1000UL);
232
233     sessaoEmAndamento = false;
234     sessoesConcluidas++;
235 }
236
237 static void processarSessao() {
238     if (!tempoAlcancado(tProximaLeitura)) return;
239
240     leituras[indiceLeitura] = gerarLeitura();
241     Serial.printf("Leitura %u: %d cm\n", (unsigned)(indiceLeitura + 1), leituras[indiceLeitura]);
242
243     indiceLeitura++;
244     tProximaLeitura += INTERVALO_LEITURA_MS;
245
246     if (indiceLeitura >= LEITURAS_POR_SESSAO) finalizarSessao();

```

```

247 }
248
249 /* =====
250 COMANDOS DE TESTE PELO SERIAL (testes 6, 7 e 8 de forma deterministica)
251 ===== */
252 static void executarSessaoDeTeste(const int valores[], const char *rotulo) {
253     Serial.println();
254     Serial.println("-----");
255     Serial.printf ("SESSAO DE TESTE INJETADA - %s\n", rotulo);
256     Serial.println("-----");
257     for (uint8_t i = 0; i < LEITURAS_POR_SESSAO; i++) {
258         Serial.printf("Leitura %u: %d cm\n", (unsigned)(i + 1), valores[i]);
259     }
260     processarResultadosDaSessao(valores, LEITURAS_POR_SESSAO);
261 }
262
263 static void processarComandosSeriais() {
264     if (!Serial.available()) return;
265
266     String cmd = Serial.readStringUntil('\n');
267     cmd.trim();
268     cmd.toUpperCase();
269     if (cmd.length() == 0) return;
270
271     if (cmd == "T1") {
272         const int v[LEITURAS_POR_SESSAO] = {18, 16, 17, 20, 16}; /* mediana 17 */
273         executarSessaoDeTeste(v, "T1: mediana >= 16 cm");
274     } else if (cmd == "T2") {
275         const int v[LEITURAS_POR_SESSAO] = {13, 15, 20, 15, 12}; /* mediana 15 */
276         executarSessaoDeTeste(v, "T2: 14 cm < mediana < 16 cm");
277     } else if (cmd == "T3") {
278         const int v[LEITURAS_POR_SESSAO] = {10, 14, 13, 12, 14}; /* mediana 13 */
279         executarSessaoDeTeste(v, "T3: mediana <= 14 cm");
280     } else {
281         Serial.printf("Comando desconhecido: \"%s\". Use T1, T2 ou T3.\n", cmd.c_str());
282     }
283 }
284
285 /* =====
286 VERIFICACAO DE VERSAO NO BOOT
287 Comprova a situacao "a versao instalada ja e a mais recente" (secao 13).
288 ===== */
289 static String extrairValorJson(const String &json, const String &chave) {
290     String alvo = "\"" + chave + "\"";
291     int p = json.indexOf(alvo);
292     if (p < 0) return "";
293     p = json.indexOf(':', p + alvo.length());
294     if (p < 0) return "";
295     int ini = json.indexOf('"', p);
296     if (ini < 0) return "";
297     int fim = json.indexOf('"', ini + 1);
298     if (fim < 0) return "";
299     return json.substring(ini + 1, fim);
300 }
301
302 static int versaoParaNumero(const String &versao) {
303     int ponto = versao.indexOf('.');
304     int maior = (ponto < 0) ? versao.toInt() : versao.substring(0, ponto).toInt();
305     int menor = (ponto < 0) ? 0 : versao.substring(ponto + 1).toInt();
306     return maior * 100 + menor;
307 }
308
309 static bool conectarWiFi() {
310     if (WiFi.status() == WL_CONNECTED) return true;
311
312     Serial.printf("[OTA] Conectando a rede \"%s\"...\n", WIFI_SSID);
313     WiFi.mode(WIFI_STA);
314     WiFi.begin(WIFI_SSID, WIFI_PASS, WIFI_CANAL);
315
316     uint32_t limite = millis() + WIFI_TIMEOUT_MS;
317     while (WiFi.status() != WL_CONNECTED && !tempoAlcancado(limite)) {
318         delay(250);
319         Serial.print(".");
320     }
321     Serial.println();
322
323     if (WiFi.status() != WL_CONNECTED) {
324         Serial.println("[OTA] ERRO: nao foi possivel conectar ao Wi-Fi.");
325         Serial.println("[OTA] O no segue medindo normalmente com o FW 2.0.");
326         return false;
327     }
328
329     Serial.printf("[OTA] Wi-Fi conectado. IP: %s\n", WiFi.localIP().toString().c_str());
330     return true;

```

```

331 }
332
333 static void verificarVersaoInstalada() {
334     Serial.println();
335     Serial.println("-----");
336     Serial.println("VERIFICACAO DE VERSAO APOS O BOOT");
337     Serial.printf ("Versao instalada: %s\n", FW_VERSION);
338     Serial.println("-----");
339
340     if (!conectarWiFi()) return;
341
342     WiFiClientSecure cliente;
343     cliente.setInsecure();
344     cliente.setTimeout(20000);
345
346     HTTPClient http;
347     if (!http.begin(cliente, MANIFEST_URL)) {
348         Serial.println("[OTA] ERRO: URL do manifesto invalida.");
349         return;
350     }
351     http.setFollowRedirects(HTTPC_STRICT_FOLLOW_REDIRECTS);
352     http.setTimeout(20000);
353
354     int codigo = http.GET();
355     if (codigo != HTTP_CODE_OK) {
356         Serial.printf("[OTA] ERRO: manifesto inacessivel (HTTP %d - %s).\n",
357             codigo, http.errorToString(codigo).c_str());
358         http.end();
359         return;
360     }
361
362     String corpo = http.getString();
363     http.end();
364
365     String versaoRemota = extrairValorJson(corpo, "version");
366     Serial.printf("[OTA] Versao instalada: %s | versao disponivel: %s\n",
367         FW_VERSION, versaoRemota.c_str());
368
369     if (versaoParaNumero(versaoRemota) > versaoParaNumero(FW_VERSION)) {
370         Serial.println("[OTA] Existe uma versao mais nova publicada no repositorio.");
371     } else {
372         Serial.println("[OTA] Firmware ja esta na versao mais recente. Nenhuma acao necessaria.");
373     }
374 }
375
376 /* =====
377     SETUP / LOOP
378     ===== */
379 void setup() {
380     Serial.begin(115200);
381     Serial.setTimeout(50);
382     delay(300);
383
384     configurarLed();
385     estadoAtual = ESTADO_NORMAL;
386     atualizarLedPorEstado();
387
388     randomSeed(esp_random());
389
390     Serial.println();
391     Serial.println("#####");
392     Serial.printf (" PROJETO MOTIVA - FIRMWARE %s\n", FW_VERSION);
393     Serial.println(" Atualizacao OTA concluida com sucesso");
394     Serial.println(" Novos recursos: ordenacao, mediana e histerese");
395     Serial.println(" Comandos de teste: T1 / T2 / T3");
396     Serial.println("#####");
397
398     verificarVersaoInstalada();
399
400     tProximaSessao = millis();
401 }
402
403 void loop() {
404     processarComandosSeriais();
405
406     if (!sessaoEmAndamento && tempoAlcancado(tProximaSessao)) {
407         iniciarSessao();
408     }
409
410     if (sessaoEmAndamento) {
411         processarSessao();
412     }
413 }

```

14. Instrucoes de execucao

No Wokwi (demonstracao)

1. Abrir o projeto Wokwi: <https://wokwi.com/projects/475816561802161153>
2. Clicar em **Play**. O LED acende **azul** e o Serial Monitor mostra o FW 1.0.
3. Aguardar **3 sessoes completas** (~104 s). O no conecta ao Wi-Fi, consulta o manifesto, detecta a versao 2.0 e inicia o download (LED **magenta**).
4. Apos a gravacao o ESP32 reinicia sozinho no **Firmware 2.0**, e o LED passa a indicar o estado (**verde** NORMAL / **vermelho** ALERTA).
5. Para testar a histerese de forma deterministica, digitar no Serial Monitor: T1 (mediana ≥ 16), T2 (mediana = 15), T3 (mediana ≤ 14).

Compilando localmente

```
brew install arduino-cli
arduino-cli config init
arduino-cli config add board_manager.additional_urls \
  https://espressif.github.io/arduino-esp32/package_esp32_index.json
arduino-cli core update-index
arduino-cli core install esp32:esp32
./build.sh
```

O `build.sh` compila as duas versoes, gera o `firmware_v2.bin`, imprime tamanho e SHA-256 e aborta se o binario nao couber na particao de aplicacao.

Comandos de teste do Firmware 2.0

Comando	Leituras injetadas	Mediana	Resultado esperado
T1	18 16 17 20 16	17 cm	-> ALERTA (vermelho)
T2	13 15 20 15 12	15 cm	mantem o estado anterior
T3	10 14 13 12 14	13 cm	-> NORMAL (verde)

A sequencia T1 → T2 → T3 → T2 demonstra a histerese **nos dois sentidos**: o T2 mantem ALERTA quando vem depois do T1, e mantem NORMAL quando vem depois do T3. E o mesmo valor de mediana produzindo estados diferentes — que e exatamente o que histerese significa.